VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Course CO3021 — Database Management Systems



DATABASE MANAGEMENT SYSTEMS PROJECT REPORT

Clinical Case-Study: Low Back Pain Diagnosis & Imaging Pipeline

---

Workload-Driven Evaluation of Microsoft SQL Server 2025:

Storage Architectures, Relational Ingestion, and AI Feature Extraction Pipeline (Role 1)

Applied to 48,345 Lumbar Spine MRI Scans and Radiologist Clinical Reports

---

Group 02

Course Instructor: PhD. Phan Trong Nhan

Project Repository: https://github.com/qmarcusv/DBMS_TA_Group2

Ho Chi Minh City, October 2026

Contents

1 Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1) 1

1.1 Clinical Problem Context and Dataset Overview . . . 1

1.2 The Three Canonical Clinical Queries Contract . . . 1

1.3 Theoretical Foundations of Database Storage . . . 2

1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies . . . 2

1.4.1 Architectural Comparison of Storage Approaches . . . 2

1.4.2 Architectural Rationale for Relative Path Selection . . . 2

1.5 Schema Architecture and Relational Integrity Constraints . . . 2

1.6 Comparative Evaluation of Data Ingestion Pipelines . . . 3

1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU . . . 4

1.7.1 Pipeline Architecture and CPU Parallelization . . . 4

1.7.2 Mathematical Similarity Metric in T-SQL . . . 4

1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs . . . 4

1.8 Analysis Across All Three Canonical Queries . . . 4

Chapter 1: Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1)

1.1 Clinical Problem Context and Dataset Overview

Modern Picture Archiving and Communication Systems (PACS) and Radiology Information Systems (RIS) manage heterogeneous medical data comprising structured diagnostic records and voluminous medical imagery. This project addresses the clinical workflow of diagnosing Low Back Pain (LBP) and Lumbar Spinal Stenosis (LSS) using the Mendeley Clinical Lumbar Spine MRI dataset:

- **Radiologist Clinical Notes:** 575 clinical narrative reports detailing lumbar pathology including disc herniation, annular tears, thecal sac compression, and neural foramen stenosis.
- **Multimodal MRI Scans:** 48,345 DICOM slices (`.ima`) distributed across 515 patients in 4-to-5 level nested directory structures covering sagittal (T1, T2) and axial sequences.

1.2 The Three Canonical Clinical Queries Contract

Following the instructor's updated evaluation directives, all sub-roles evaluate and interpret DBMS behavior across the exact three canonical clinical queries engineered by Role 1:

Query 1 (Q1) — Point Lookup and One-to-Many Clinical Join

Retrieves the complete narrative diagnostic report alongside all associated MRI scan paths for a designated patient:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001';
```

Query 2 (Q2) — Sequence-Selective Diagnostic Filtering

Retrieves imaging slices restricted to sagittal T2-weighted pulse sequences (`T2_TSE_SAG`) frequently utilized for spinal cord and disc herniation assessment:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001' AND s.SeriesDescription LIKE '%T2%SAG%';
```

Query 3 (Q3) — Content-Based Image Retrieval via Vector Similarity

Calculates mathematical proximity (Euclidean distance and cosine similarity) between an index query slice ($ScanID = 1$) and all other scans in the database using high-dimensional feature embeddings:

```
DECLARE @TargetScanID INT = 1;
DECLARE @T1 FLOAT, @T2 FLOAT, @T3 FLOAT, @T4 FLOAT;
SELECT @T1 = V_Dim1, @T2 = V_Dim2, @T3 = V_Dim3, @T4 = V_Dim4
FROM Image_Features WHERE ScanID = @TargetScanID;

SELECT TOP 3 s.ScanID, s.PatientID, s.SeriesDescription, s.ImagePath,
    SQRT(SQUARE(f.V_Dim1 - @T1) + SQUARE(f.V_Dim2 - @T2) +
         SQUARE(f.V_Dim3 - @T3) + SQUARE(f.V_Dim4 - @T4)) AS EuclideanDistance
FROM Image_Features f
INNER JOIN MRI_Scans s ON f.ScanID = s.ScanID
WHERE f.ScanID <> @TargetScanID
ORDER BY EuclideanDistance ASC;
```

1.3 Theoretical Foundations of Database Storage

In relational database theory, storage subsystems organize records into fixed-size physical blocks or pages. In Microsoft SQL Server, the fundamental unit of I/O is the **8KB Data Page** (8,192 bytes), of which 8,060 bytes accommodate user data after subtracting page headers and slot arrays. When storing binary large objects (BLOBs) exceeding 8KB, conventional row architectures fragment records across off-row text/image tree allocations. This architectural reality presents critical performance tradeoffs when designing clinical multimedia databases.

1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies

To handle 13GB of DICOM MRI scans, Microsoft SQL Server provides four distinct architectural mechanisms:

1.4.1 Architectural Comparison of Storage Approaches

1. **In-Row and Out-of-Row `VARBINARY(MAX)`:** Binary bytes reside directly within the `.mdf` database file. Slices larger than 8KB migrate to dedicated LOB pages. *Drawback:* Loading medical images into memory causes severe **Buffer Pool thrashing**, displacing critical index and table pages from RAM cache and inflating backup archives to multi-gigabyte sizes.
2. **FILESTREAM Storage:** Combines the SQL Server Database Engine with the Windows NTFS/ReFS file system. Unstructured files reside in operating system directories while remaining under SQL Server transactional control. *Mechanism:* Data transfers occur via native Win32 streaming APIs, **completely bypassing the SQL Server Buffer Pool** and preserving cache memory for relational query processing.
3. **FileTable Technology:** Built upon FILESTREAM, FileTable integrates a Windows Win32 namespace hierarchy. Operating system users and external PACS systems can drag, drop, and edit `.ima` files via standard Windows Network Shares (SMB), while SQL Server automatically reflects changes within relational tables.
4. **Relative File Path Reference (`NVARCHAR(500)`):** The database stores only relative filesystem paths pointing to operating system directories. The physical binary assets remain completely decoupled from the relational engine.

Table 1.1: Comprehensive Comparison of Large Binary (BLOB) Storage Architectures in MSSQL

Evaluation Metric	VARBINARY(MAX)	FILESTREAM	FileTable	Relative Path (Selected)
Physical Location	Database `.mdf` file	NTFS/ReFS directories	NTFS via SMB share	External file directory
Buffer Pool Cache	Heavy RAM pollution	Bypasses Buffer Pool	Bypasses Buffer Pool	Zero Buffer Pool footprint
ACID Transactionality	Full native logging	Full Win32 transactional	Full Win32 transactional	Application-level control
Backup Archive Size	13GB+ database backup	Included in backup stream	Included in backup stream	Compact 53MB metadata backup
AI Framework Access	Requires SQL extraction	Win32 API streaming	Direct Windows file path	Native Python `pydicom` I/O

1.4.2 Architectural Rationale for Relative Path Selection

For this research workload, the **Relative File Path** architecture was selected. Decoupling the 13GB image binaries from the database reduced the database footprint from 13GB to 53MB. This prevents Buffer Pool contention during Indexing and Query Optimization benchmarks while enabling native, multi-threaded PyTorch feature extraction on disk without database roundtrip latency.

1.5 Schema Architecture and Relational Integrity Constraints

Rather than allowing SSMS Import Wizards to infer loose schemas defaulting to `NVARCHAR(MAX)`, strict DDL constraints were implemented to optimize data page packing and enforce relational integrity:

```
CREATE TABLE Patients (
    PatientID VARCHAR(20) NOT NULL,
    PatientName NVARCHAR(100) NULL,
    ClinicalNotes NVARCHAR(MAX) NULL,
```

```
    DiagnosisStatus VARCHAR(20) DEFAULT 'Confirmed',
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT PK_Patients PRIMARY KEY CLUSTERED (PatientID)
);

CREATE TABLE MRI_Scans (
    ScanID INT IDENTITY(1,1) NOT NULL,
    PatientID VARCHAR(20) NOT NULL,
    StudyInstanceUID VARCHAR(150) NULL,
    SeriesDescription NVARCHAR(255) NOT NULL,
    ImagePath NVARCHAR(500) NOT NULL,
    AcquisitionDate DATETIME DEFAULT GETDATE(),
    CONSTRAINT CK_Series_Valid CHECK (SeriesDescription NOT LIKE '%INVALID%'),
    CONSTRAINT PK_MRI_Scans PRIMARY KEY CLUSTERED (ScanID),
    CONSTRAINT FK_MRI_Scans_Patients FOREIGN KEY (PatientID)
        REFERENCES Patients(PatientID) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Image_Features (
    FeatureID INT IDENTITY(1,1) NOT NULL,
    ScanID INT NOT NULL UNIQUE,
    FeatureVector NVARCHAR(MAX) NOT NULL,
    V_Dim1 FLOAT NULL, V_Dim2 FLOAT NULL, V_Dim3 FLOAT NULL, V_Dim4 FLOAT NULL,
    CONSTRAINT PK_Image_Features PRIMARY KEY CLUSTERED (FeatureID),
    CONSTRAINT FK_Features_Scans FOREIGN KEY (ScanID) REFERENCES MRI_Scans(ScanID) ON DELETE
    CASCADE
);
```

Relational Integrity and Constraint Highlights

- **Compact Primary Keys:** `PatientID VARCHAR(20)` ensures high page density within Clustered B-Tree leaf pages compared to unrestricted unicode datatypes.
- **Transaction Test Constraint:** The check constraint `CK_Series_Valid` deliberately rejects records containing `'INVALID'` strings, establishing the deterministic error boundary required by Role 4 to verify atomic transaction rollback.

1.6 Comparative Evaluation of Data Ingestion Pipelines

Bulk data loading was evaluated across four industry standard ingestion methodologies:

Table 1.2: Comparative Benchmark of Data Ingestion Methodologies into SQL Server

Ingestion Method	Mechanism	Logging Mode	Ingestion Speed	Automation Support
SSMS Flat File Wizard	Interactive GUI	Fully Logged	15–20 seconds	Manual interaction
T-SQL BULK INSERT	In-process Server Worker	Minimally Logged (`TABLOCK`)	1.2 seconds	Fully scriptable
Command-Line bcp	External client utility	Minimally Logged (`-h TABLOCK`)	1.1 seconds	CLI and batch files
Python pyodbc	Client cursor array	Fully Logged	8–10 seconds	Integrated AI pipeline

Identity Column Mapping Resolution via View Abstraction

During direct `BULK INSERT` execution into tables containing an auto-incrementing identity column (`ScanID INT IDENTITY`), SQL Server raises error `Msg 7301` due to column mapping mismatches between the 5-column CSV and 6-column physical schema. This was resolved without complex XML format files by creating an updatable relational view:

```
CREATE VIEW dbo.vw_MRI_Scans_Import AS
SELECT PatientID, StudyInstanceUID, SeriesDescription, ImagePath, AcquisitionDate
FROM dbo.MRI_Scans;
```

```
BULK INSERT dbo.vw_MRI_Scans_Import FROM '...clean_mri_scans.csv'
WITH (FIRSTROW = 2, FORMAT = 'CSV', FIELDQUOTE = '"', FIELDTERMINATOR = ',',
      ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
DROP VIEW dbo.vw_MRI_Scans_Import;
```

1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU

To support Query 3 (Content-Based Medical Image Retrieval), an end-to-end AI feature extraction pipeline was engineered using PyTorch and `pydicom`:

1.7.1 Pipeline Architecture and CPU Parallelization

1. **DICOM Matrix Ingestion:** Medical slices (`.ima`) are parsed via `pydicom.dcmread`, normalized from 12-bit dynamic range ($[0, 4095]$) to $[0, 255]$ uint8 arrays, and resized to $224 \times 224 \times 3$.
2. **Pretrained ResNet-18 Inference:** Slices are processed in minibatches of 64 through an ImageNet-pretrained ResNet-18 model. The final fully connected classification layer is truncated, extracting 512-dimensional feature embeddings from the penultimate `AdaptiveAvgPool2d` layer.
3. **Vector Normalization and Database Persistence:** Each 512-dimensional embedding $\mathbf{v}$ is normalized via Euclidean norm ($\|\mathbf{v}\|_2 = 1$). The leading dimensions are persisted in float columns (`V_Dim1..V_Dim4`) alongside full JSON vector strings.

Throughput Performance on 20-Core CPU

By leveraging batch inference across 20 CPU threads, the pipeline achieved an average throughput of **20.2 images/second**, extracting all 48,345 slices in **39.9 minutes** without requiring specialized GPU hardware.

1.7.2 Mathematical Similarity Metric in T-SQL

In Query 3, similarity is evaluated using Euclidean Distance:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{k=1}^{4} (u_k - v_k)^2} \tag{1.1}$$

Because feature vectors are L_2-normalized, Euclidean distance relates monotonically to Cosine Similarity ($\|\mathbf{u} - \mathbf{v}\|_2^2 = 2 - 2\cos(\theta)$). Executing Query 3 over all 48,345 scans yielded the top-3 similar slices with sub-100ms execution latency (98ms elapsed time).

1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs

While Microsoft SQL Server executes vector distance calculations cleanly via arithmetic operators (`Compute Scalar`) and memory-bounded sorts (`Top N Sort`), it fundamentally lacks Approximate Nearest Neighbor (ANN) indexing structures such as HNSW (Hierarchical Navigable Small World) or IVF (Inverted File Index). Consequently, vector search in standard RDBMS remains an $\mathcal{O}(N)$ brute-force table scan. In large-scale clinical production, dedicated vector stores (e.g., Milvus, Qdrant) or extensions (`pgvector`) are recommended for billion-scale embeddings.

1.8 Analysis Across All Three Canonical Queries

The engineered data and storage architecture empowers all three clinical query contracts:

- **Impact on Query 1 (Point Lookup):** Returning text paths rather than binary BLOBs prevents network saturation and eliminates Buffer Pool contention.
- **Impact on Query 2 (Sequence Filtering):** Strict `NVARCHAR(255)` sequence typing allows the Query Optimizer to leverage B-Tree index seeks on pulse sequence descriptors without reading external storage.
- **Impact on Query 3 (Similarity Search):** Native float representation enables direct relational calculation of Euclidean distances, seamlessly integrating machine learning embeddings into SQL workflows.